

# W38\_live\_coding

Lau Møller Andersen

2025-09-16

Example: using categorical instead of continuous

```
library(lme4)
data('sleepstudy')

inelegant.single <- lm(Reaction ~ Days, data=sleepstudy)
plot(Reaction ~ Days, data = sleepstudy)
abline(inelegant.single)
factorial.model <- lm(Reaction ~ factor(Days), data=sleepstudy)
elegant.multi <- lmer(Reaction ~ Days + (Days | Subject), data=sleepstudy)
```

Example: aggregating proportions

```
library(lme4)
set.seed(7)
n.sub <- 10
n.trials <- 10
correct.rate <- 0.97

n.successes <- rbinom(n.sub, n.trials, correct.rate)

## aggregate
props <- data.frame('proportions' = n.successes / n.trials)

aggregated.model <- lm(proportions ~ 1, data=props)
print(confint(aggregated.model))

##          2.5 %   97.5 %
## (Intercept) 0.8892064 1.010794

## glmer
library(boot)
successes <- numeric(length=n.sub * n.trials)

for(sub in 1:n.sub)
{
  first.index <- ((sub-1) * 10) + 1
  success.sub <- n.successes[sub]
  successes[first.index:(first.index+n.successes[sub]-1)] <- 1
}

single.trials <- data.frame('successes' = successes, subject=rep(1:n.sub, each=n.trials))
```

```
right.model <- glmer(successes ~ 1 + (1 | subject), data=single.trials, family='binomial')
inv.logit(confint(right.model))
```

```
##                2.5 %    97.5 %
## .sig01         0.5000000 0.9997431
## (Intercept) 0.9026412 0.9999230
```

Example: missing data

```
library(lme4)
data('sleepstudy')
sleepstudy.NA <- sleepstudy
set.seed(35)
NA.indices <- sample(1:dim(sleepstudy.NA)[1], 75)
sleepstudy.NA$Reaction[NA.indices] <- NA
single.level.model <- lm(Reaction ~ Days + Subject - 1, data=sleepstudy.NA)
multilevel.model <- lmer(Reaction ~ Days + (1 | Subject), data=sleepstudy.NA)

subs <- unique(sleepstudy.NA$Subject)
n.sub <- length(subs)
n.data.points <- numeric(length=n.sub)

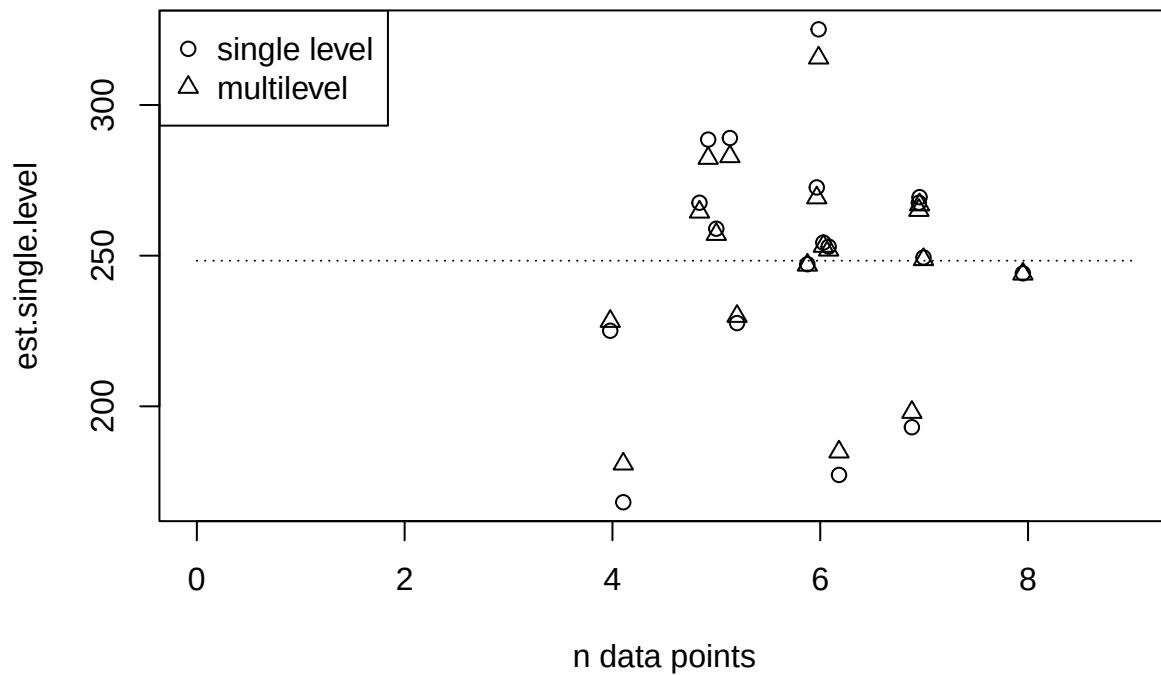
for(sub.index in 1:n.sub)
{
  n.data.points[sub.index] <- sum(!is.na(sleepstudy.NA$Reaction
                                         [sleepstudy.NA$Subject == subs[sub.index]]))
}
## single level estimates
est.single.level <- single.level.model$coefficients[2:(n.sub+1)]

jittered <- jitter(n.data.points)

plot(jittered, est.single.level, pch=1, xlim=c(0, 9), xlab='n data points')
est.average <- fixef(multilevel.model)[1]
lines(c(0, 9), rep(est.average, 2), lty=3)

## multilevel estimates

est.multilevel <- ranef(multilevel.model)$Subject + est.average
points(jittered, t(est.multilevel), pch=2)
legend('topleft', c('single level', 'multilevel'), pch=1:2)
```



Example: covariance in predictors

```
get.SIGMA <- function(model) sigma(model)^2 * (getME(model, 'Lambda') %*% getME(model, 'Lambdat'))

for(seed in c(1, 7))
{
  print(paste('Random seed is:', seed))
  set.seed(seed)
  sleepstudy$Days2 <- 2 * sleepstudy$Days + rnorm(length(sleepstudy$Days))
  model <- lmer(Reaction ~ Days + Days2 + (Days + Days2 | Subject), data=sleepstudy)
  SIGMA <- get.SIGMA(model)
  print(paste('N columns is:', dim(SIGMA)[2]))
  print(paste('Estimated rank is:', rankMatrix(SIGMA)[1]))
}

## [1] "Random seed is: 1"
## boundary (singular) fit: see help('isSingular')
## [1] "N columns is: 54"
## [1] "Estimated rank is: 36"
## [1] "Random seed is: 7"
## Warning in checkConv(attr(opt, "derivs"), opt$par, ctrl = control$checkConv, :
## Model failed to converge with max|grad| = 0.0433554 (tol = 0.002, component 1)
## [1] "N columns is: 54"
## [1] "Estimated rank is: 54"
```